

认证 workflow：保护智能体 AI 的系统方法

Mohan Rajagopalan
MACAW Security, Inc.
mohan@macawsecurity.com

Vinay Rao
ROOST.tools
vinay@roost.tools

摘要——智能体 AI 系统能自动化企业 workflow，但现有防御手段——护栏 (Guardrails)、语义过滤器——都是概率性的，且经常被绕过。本文提出认证 workflow (Authenticated Workflows)，即面向企业智能体 AI 的首个完整信任层。安全问题可归约为保护四个基本边界：提示 (Prompt)、工具 (Tool)、数据 (Data) 和上下文 (Context)。我们在每个边界跨越点强制执行意图 (操作满足组织策略) 和完整性 (操作具有密码学真实性)，将攻击类的密码学消除与运行时策略执行相结合，提供确定性安全——操作要么携带有效密码学证明，要么被拒绝。我们引入 MAPL，一种 AI 原生策略语言，能在智能体演化和调用上下文变化时动态表达智能体约束，通过层级组合与工作流依赖的密码学证言 (Attestation)，策略规模为 $O(\log M + N)$ ，而传统规则为 $O(M \times N)$ 。我们通过一个通用安全运行时验证了实用性，该运行时通过薄适配器 (无需修改协议) 集成了九个主流框架 (MCP、A2A、OpenAI、Claude、LangChain、CrewAI、AutoGen、LlamaIndex、Haystack)。形式化证明建立了完备性和可靠性。实验验证显示，在 174 个测试用例中达到 100% 召回率且零误报，防护 OWASP Top 10 中的 9 项风险，并完全缓解了两个高影响力的生产环境 CVE。

1. 引言

企业正苦于将智能体 AI (agentic AI) 系统部署到生产环境。这些系统承诺能自动化复杂 workflow——管理金融交易、病历记录和关键基础设施——但它们引入的安全挑战是现有防御手段无法应对的。例如，在发布 OpenAI Atlas 浏览器后数小时内，研究人员就演示了网页内容中嵌入的恶意指令可触发助手泄露凭证 [1]。OpenAI 的 CISO 承认“提示注入 (prompt injection) 仍然是一个未解决的问题” [2]。

挑战远不止提示注入本身。LLM 无法区分指令与数据。非确定性执行路径无法预测或静态分析。多轮交互允许渐进式上下文操控，其中每条消息看似无害。现有方法基于模式查找或概率判断——两者都需要枚举攻击，使防御者困于未知之未知。

本方案通过系统化方法消除未知之未知：不是枚举攻击，而是确定性界定系统。核心洞察是：智能体系统可抽象为简单、拜占庭式的分布式系统——多个实体在明确界定的边界上交互。安全问题遂归结为保护边界穿越。概念上，本方案基于在每个边界穿越处满足两个属性：理解意图 (intent) 以确保操作满足组织策略，强制执行完整性 (integrity) 以确保操作真实且未被篡改。两者缺一不可，单独任意一个都不充分。此设计将智能体 workflow 约简为确定性分布式系统，其中边界由策略守卫，穿

越由密码学保护。攻破系统需要攻破密码学，而非精心构造提示。多类攻击——如身份伪造、会话重放、策略替换等——在设计层面 (by-design) 即被消除：它们必须攻破密码学原语才能得逞；其他攻击——如未授权数据访问、权限提升、凭证泄露等——则在运行时由策略阻断。架构上，策略层面强制执行 (by-policy enforcement) 与设计层面消除 (by-design elimination) 共同提供互补、可组合的纵深防御。

我们研究了 100+ 个智能体应用，识别出关键挑战：

(1) 四个并发攻击面——工具、数据、提示、上下文——攻击可跨面组合，达成任意单一表面无法实现的目标。(2) 异构框架——智能体构建使用多种协议 (MCP、A2A)、LLM 接口 (Claude、OpenAI)、编排器 (LangChain、CrewAI、AutoGen)。异构性是生态系统的持久特征。(3) 组合式安全缺口——逐框架检查会遗漏跨越组合的违规。(4) 开发者负担——要求在异构框架的每一次交互中嵌入安全逻辑是不现实的。

综合来看，攻击面的增长速度超过工具的应对速度，凸显出一个关键缺口：面向 AI 的信任层 (trust layer)——位于应用层防御 (不可组合) 与基础设施原语 (缺乏 workflow 语义) 之间的安全基础设施。

为解决 (1)，我们引入认证 workflow (authenticated workflows)——一种协议级原语，在每次交互中强制执行意图与完整性。我们证明了四个攻击面是最小且完备的。协议级定位解决了异构性 (2) 并消除组合式缺口 (3)。面向 9 种主流框架的薄封装层解决了开发者负担 (4)，为企业 AI 信任层奠定基础。

认证 workflow 结合密码学、零信任 (zero-trust) 原则和运行时策略强制执行，确保每个操作真实、授权且可验证。我们提出一种新颖设计：使用嵌入各控制面 (control surface) 的分布式策略执行点 (Policy Enforcement Point, PEP)，独立验证密码学证明，以亚毫秒级开销运行，无需集中式基础设施。每个表面对操作独立验证后再执行，形成纵深防御——攻破一个表面不会攻破其他表面。

为指定意图，我们引入 MAPL——一种新的 AI 原生策略语言，使用户能以动态、可扩展的方式表达智能体约束，即使智能体不断演进且调用上下文持续变化。MAPL 提供三项关键能力：可组合语法与继承使策略能通过交集语义 (intersection semantics) 分层组合；极简约通过层级组合 (hierarchical composition) 与隐式主体 (principal) 解析将规模从 $O(M \text{ 用户} \times N \text{ 资源})$ 降至 $O(\log M + N)$ ；以及基于密码学证言 (attestation) 的动态状态——证明操作已完成的签名声明，使策略得以表达如“仅在匿名化完成证言存在后才能导出数据”，并具

备密码学强制力。这些进步克服了传统策略系统（OPA、AWS Cedar）在智能体环境中的局限。

本文聚焦跨操作边界的控制流保护——工具调用与数据检索——与 Rajagopalan 等人 [3] 的研究互补，后者保护流经提示与上下文的数据。二者结合，为企业 AI 提供了形式化的信任层。通过一个支撑九种框架（第 5 节）的通用安全运行时，以薄适配器（200-500 LOC）实现，无需协议变更——证明了框架无关的安全能力。

形式上，我们证明了四个边界是完备且最小的，分布式强制执行是可靠的，策略组合保持安全属性。实验上，通过全面的攻击测试和生产 CVE 分析验证了这些属性。

当前智能体防御——护栏（guardrail）、提示过滤器、语义分析（模式匹配、ML 模型、启发式）——注重精确率但召回率低甚至为零。对抗性提示、编码技巧和新型模式通过语义盲区持续绕过它们，而误报则阻断合法操作。与之对比，我们的密码学强制执行保证高精确率和高召回率：操作要么携带满足策略的有效签名，要么被直接拒绝。攻破系统需要击碎密码学原语——而非构造巧妙模式。这实现了完整的违规检测、零误报，以及基于计算困难性的确定性信任。

贡献。我们提出认证工作流——一种协议级原语，在四个控制面（提示、工具、数据、上下文）上强制执行密码学验证，并证明其完备且最小（引理 6）。我们引入 MAPL，一种 AI 原生策略语言，通过密码学证言提供密码学工作流依赖，并通过层级组合将策略规约从 $O(M \times N)$ 降至 $O(\log M + N)$ （定理 1-3）。我们展示了在九种异构框架（MCP、A2A、OpenAI、Claude、LangChain、CrewAI、AutoGen、LlamaIndex、Haystack）上的通用部署，通过薄适配器（200-500 LOC）实现，无需任何协议修改，分布式策略执行点嵌入每个边界提供零信任验证，开销为亚毫秒级。形式化证明确立了完备性与可靠性（引理 1-7）；实验验证展示了确定性保证（100% 召回率，0% 误报），覆盖 174 个测试用例，涵盖 10 项 OWASP Top 10 风险中的 9 项，并完全缓解两个生产 CVE（OpenAI Atlas、GitHub MCP）。

2. 问题：智能体如何工作

为理解保障智能体系统安全的难点，考虑图 1，它展示了调用“读取 Q4 财务文档并将摘要发送给 john@company.com”时涉及的实体。用户提示携带指令流向 LLM。LLM 推理后决定调用文件系统读取和邮件发送操作——MCP 工具、OpenAI 函数调用或自定义实现。这些工具调用从文档存储中检索数据。当智能体执行此多轮工作流时，它维护着跟踪交互历史的应用级状态。

这些交互穿越四个基本边界。**S1——提示 (Prompts)** 将指令带入 LLM，引导推理与工具选择。**S2——工具 (Tools)** 执行特权操作——文件系统访问、数据库查询、API 调用、邮件发送。**S3——数据 (Data)** 从外部数据源流入智能体推理——文档存储、RAG 语料库、向量数据库、网页抓取。**S4——上下文 (Context)** 在多轮交互中维护对话状态。一个根本挑战：LLM 以相同方式处理提示和文档内容——Q4.pdf 中的恶意指令与用户请求不可区分。系统运行在隐式信任之上：上下文默认有效，工具默认已授权，数据源默认良性。这些边界是框架无关的——LangChain 链和 CrewAI 团队均表现为 prompt→LLM→tool 序列，全部穿越相同的四个表面。这种普遍性使协议级安全能跨异构系统统一生效。

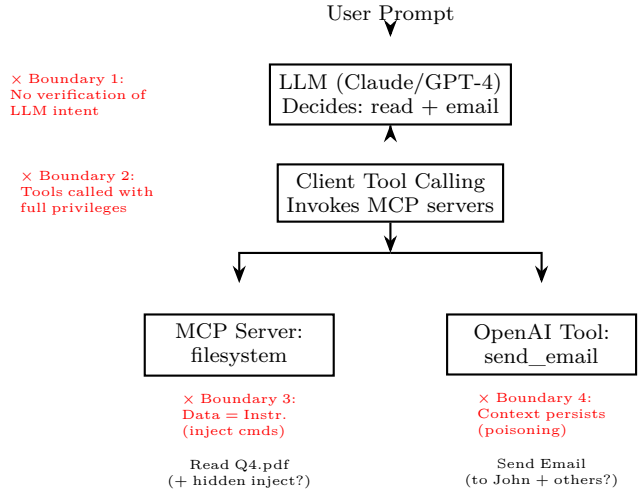


图 1. 智能体 workflows 中的跨四边界攻击级联。

任一表面都可作为攻击入口点，其后果跨其他表面级联扩散。在 OpenAI 的 Atlas 攻击 [1] 中，嵌入数据（S3）的恶意指令流入 LLM 推理（S1），生成工具调用（S2）以泄露凭证，全部在上下文（S4）中被记录为正常执行。不同攻击从不同表面进入。通过以下威胁模型进行形式化。

威胁模型。假设一个复杂敌手，通过敌手能力和信任假设对威胁模型进行形式化。

敌手能力 A1——应用控制：敌手可控制应用层代码。**A2——内容注入：**将恶意内容注入数据源、提示或上下文。**A3——组件妥协：**可妥协组件并获取私钥。**A4——网络攻击：**拦截、重放或篡改网络流量。**A5——组合式攻击：**跨交互执行渐进式攻击或跨越框架边界，其中每个操作看似已授权，但组合起来违反策略。

信任假设 L1——密码学困难性：密码学原语提供计算困难性——敌手无法在无私钥情况下伪造签名。**L2——可信控制平面：**控制平面服务（智能体注册中心、策略存储、日志、路由）运行在最小可信计算基中，具备密码学完整性保证（审计日志的哈希链（hash chain）、策略存储的 Merkle 树）及管理访问控制。**L3——强制执行完整性：**PEP 验证逻辑在框架边界处正确执行。具备应用控制（A1）的敌手可操控业务逻辑，但无法绕过框架 API 或破坏 PEP 内存——在企业部署中，框架不可变且操作通过插桩 API 路由。绕过 PEP 需要破坏框架二进制文件——等价于内核级妥协，不在本文范围内。这类似于操作系统内核安全。系统级攻击（调试器、内存损坏）不在范围内——即使此类攻击绕过本地 PEP，远端仍独立验证调用，将敌手限制在其应用范围内而不获得新权限。

安全目标针对此威胁模型，建立五项安全目标：**O1——完整性：**操作真实且未被篡改。**O2——策略强制执行：**所有操作满足组织策略。**O3——权限非提升：**组合策略不能授予比单独策略更广泛的权限。**O4——上下文完整性：**会话状态跨交互防篡改。**O5——可问责性：**所有操作具备不可否认（non-repudiation）的审计轨迹。第 7 节证明认证工作流在假设 L1-L3 下达成 O1-O5，对抗具备能力 A1-A5 的敌手。

范围本文聚焦协议级授权：以密码学方式强制执行每个操作满足组织策略。本文提供机制——最终用户提供策略。应用级安全（例如检测恶意代码）需要领域特定语义，不在协议范围内。将所有应用视为不可信，受系统级策略约束。

当前方法是被动式的——检测模式、过滤输入、沙箱（sandbox）执行。每个新攻击变种需要新检测规则，对无界攻击空间形成枚举问题。上述敌手能力（A1-A5）针对四个攻击面：应用控制与内容注入破坏提示/工具/数据，组合式攻击利用多框架间隙。**A1、A3、A4** 使基于可观测性的方案失效。近期针对 **A2** 和 **A4** 使用定制模型的工作 [4] 是概率性的，无法确定性保证安全。

本方案在协议层面，智能体 → 工具、智能体 → LLM、智能体 → 数据操作看起来完全相同——一次边界穿越。本方案将每次穿越转换为认证工作流，受策略约束（O2），验证授权（O1、O3、O4）并在执行时生成证言（O5）。下一节 3 描述一个强制执行 O3 的策略代数。设计层面机制（密码学签名、哈希链）应对组件妥协（A3）和网络攻击（A4）。策略层面机制（边界运行时验证）应对应用控制（A1）、内容注入（A2）和渐进式攻击（A5）。第 7 节证明操作要么携带满足策略的有效密码学证明，要么在执行前被阻断——确定性保证，而非概率性检测。

回到示例攻击：每个边界由独立策略保护。即使推理被破坏，受限文档仍不可访问。组件妥协后，应用被限制在策略许可范围内。攻破系统需要同时攻破所有强制执行层——计算上不可行。

问题分解为两个子问题：如何指定治理边界的策略？如何在运行时验证完整性？引入 MAPL（第 3 节）表达意图，认证工作流（第 4 节）强制执行完整性，通过通用信任层实现（第 6-7 节）。

3. 指定意图：MAPL 策略语言

引入 MAPL，一种 AI 原生策略语言，用于指定意图。智能体系统呈现出动态性（智能体变换身份、生成子智能体、委派能力）和规模（每用户 10-100 倍增长），这些特性使传统策略引擎失效。三个特性对现有系统构成挑战：上下文相关身份（Contextual identity）——智能体根据运行时上下文代表不同主体（principal），需要动态主体解析而非静态 WHO 绑定。双视角（Dual perspective）——维护完整性需要从调用方意图和资源约束两个角度独立验证每次调用；传统策略将调用方与资源紧密耦合，一旦妥协调用方即可获取资源访问权。可验证（verifiable）工作流状态——多步工作流需要顺序强制执行，由于应用报告的状态不可信，需要某种形式的密码学证言（attestation）来验证操作 A 已完成再调用操作 B。

MAPL 通过三项设计选择应对这些挑战：(1) 主体从已认证的运行时上下文推断——实现动态身份而无需策略更新；(2) 调用方策略与资源策略独立表达，通过交集（intersection）组合——实现纵深防御而无需协调；(3) 策略引用密码学验证的证言——实现可证明的工作流依赖。三者结合，构建出完备、可证明、可组合的意图表达模型。

策略语法。MAPL（MAPL 智能体策略语言）策略遵循结构化语法，支持机器验证与自动组合：

```
Policy {  
  policy_id: <unique_identifier>,  
  extends: <parent_policy_id>,  
  resources: [<resource_patterns>],  
  denied_resources: [<denial_patterns>],  
  constraints: {  
    parameters: {<resource>: {<param>: [<patterns  
      ↪ >]}},  
    denied_parameters: {<resource>: {<param>: [<  
      ↪ patterns>]}},  
    attestations: [<required_attestation_names>]  
  }  
}
```

一条 MAPL 策略包含：policy_id 用于组合链和密码学绑定的唯一标识符，extends 层级组合的父策略引用，resources 允许的操作模式；通配符 */** 匹配单层/递归层级，denied_resources 覆盖允许的显式阻断，以及 constraints（参数、拒绝参数、模式和证言要求）。附录 A 展示了具体策略示例。

层级组合。extends 字段使策略能通过交集语义从父策略继承。这使组织策略得以分层（基础 → 部门 → 团队），其中每个子策略通过增加约束来精细化父策略，决不放松限制。当一条策略扩展另一条时，有效权限是父策略与子策略的交集——保持单调约束（monotonic restriction），如下文组合代数所形式化。

表达能力。最简语法通过四种机制即可满足需求：(1) 带通配符的层级资源表达无界命名空间；(2) 正面与负面规约表达任意布尔组合；(3) 参数约束表达参数上的任意可判定谓词；(4) 证言通过密码学状态实现顺序约束。这涵盖了组织层级、双视角防御、工作流依赖、例外模式和参数控制。

组合代数。形式化策略在运行时如何组合，并证明系统满足关键安全属性。

策略结构与操作。

一条策略 $P = (R, D, C)$ 包含：

- R ：允许的资源模式
- D ：拒绝的资源模式
- C ：操作约束（参数、证言）

策略交集 $P_1 \cap P_2 = (R', D', C')$ ，其中：

- $R' = R_1 \cap R_2$ （仅当两策略均允许时才允许）
- $D' = D_1 \cup D_2$ （任一策略禁止则拒绝）
- $C' = \text{MostRestrictive}(C_1, C_2)$ （应用最严格约束）

$\text{MostRestrictive}(C_1, C_2)$ 对数值限制取最小值，对允许模式取交集，对所需证言和拒绝模式取并集。

策略执行点在运行时组合来自组织层级和双视角的策略。这确保操作只能增加约束，决不放松。若某策略不存在，则不贡献任何约束（最宽松解释，不违反任何已声明约束）。

形式化安全属性

定义权限函数（permission function）：

$\pi(P) = \{(r, op) \mid r \notin \text{Match}(D) \wedge r \in \text{Match}(R) \wedge op \text{ satisfies } C\}$

定理 1（单调约束）：对组合 $P_0 \cap \dots \cap P_n$ ：

$\forall i, j : (i < j) \Rightarrow \pi(P_0 \cap \dots \cap P_j) \subseteq \pi(P_0 \cap \dots \cap P_i)$

证明概要：由构造， $P_{i+1} = P_i \cap P_{\text{next}}$ 。资源交集： $R_{i+1} \subseteq R_i$ 。拒绝并集： $D_{i+1} \supseteq D_i$ 。因此 $\pi(P_{i+1}) \subseteq \pi(P_i)$ 。由归纳法， $\pi(P_j) \subseteq \pi(P_i)$ 对 $i < j$ 成立。□

定理 2 (传递拒绝): 若资源 r 被任一策略拒绝, 则保持拒绝: $\exists i : r \in \text{Match}(D_i) \Rightarrow r \in \text{Match}(D_{\text{eff}})$ 证明概要: $D_{\text{eff}} = D_0 \cup D_1 \cup \dots \cup D_n$. 若存在 i 使 $r \in D_i$, 则 $r \in D_{\text{eff}}$ 由集合并集可得。□
定理 3 (无权限提升): 若基础策略 P_0 拒绝 r, 则任何组合均不授予访问: $(r \in \text{Match}(D_0)) \Rightarrow r \notin \text{Allowed}(P_{\text{eff}})$ 证明: 由定理 2, 若 $r \in \text{Match}(D_0)$, 则 $r \in \text{Match}(D_{\text{eff}})$. 由 π 的定义, 被拒绝的资源不可被允许。□

安全含义 这些定理提供形式化保证: **定理 1** 防止权限扩展——增加策略只能收窄权限; **定理 2** 确保任意层的拒绝是绝对的; **定理 3** 防止通过组合策略提升权限, 即组合不能授予被拒权限。

三者共同实现第 2 节中的策略强制执行 (O2) 和权限非提升 (O3)。即使敌手妥协组件并获取其密钥 (A3), 被妥协组件仅能在其策略许可范围内执行操作——单调约束防止权限扩展, 传递拒绝确保根拒绝传播到所有派生策略。这些属性以数学方式成立——与攻击者复杂度或 LLM 行为无关。破坏它们需要攻破密码学原语 (数字签名、哈希函数), 而非操控。**禁止任意覆盖**。MAPL 明确禁止覆盖——覆盖会破坏可证明性, 使定理 1-3 不成立。替代方案是管理员创建带时效的临时组 (类比 Unix 组/权限)。这些组通过交集组合, 保持形式化保证同时支持运维灵活性。附录 A 展示了带时效的紧急访问示例。

实现规模: 将 $O(M \times N)$ 规则降至 $O(\log M + N)$ 策略 传统策略引擎对 M 个主体和 N 个资源需要 $O(M \times N)$ 条显式规则。管理组合爆炸式增长的规则不切实际; MAPL 为企业在数千用户规模下扩展智能体提供了实用方案。MAPL 的层级组合与动态主体绑定通过镜像组织结构消除了这种爆炸——分支因子为 8-16 时, 部门数量按对数增长: $D \approx \log_{8-16}(M)$ ——远小于 M , 团队向上继承并融入层级树。资源随功能增长, 与用户数量无关。因此 MAPL 仅需 $\log(M) + N$ 条策略——一次显著且实用的缩减。

综上, MAPL 的组合代数应对了全部三项智能体需求: 通过运行时主体解析实现上下文相关身份, 通过独立策略组合实现双视角, 通过密码学证言实现可验证工作流程状态。

4. 认证 workflow

本节介绍认证 workflow (authenticated workflows)——一种协议级机制, 在每次智能体边界穿越时实现确定性验证。面对具备 A1-A5 能力的攻击者 (第 2 节), 认证 workflow 提供四项**密码学保证**, 共同实现安全目标 O1-O5:

真实性 (Authenticity) (实现 O1: 完整性): 每次被接受的调用都通过密码学方式绑定到发起调用的主体 (principal) 和所执行的操作。攻击者无法在未持有主体私钥的情况下伪造调用, 同时绑定 WHO (主体) 和 WHAT (操作)。

策略绑定 (Policy Binding) (实现 O2-O3: 策略执行、权限不可升级): 控制操作的策略通过密码学方式绑定到调用, 攻击者无法替换策略而不导致签名失效。

篡改证据 (Tamper Evidence) (实现 O4: 上下文完整性): 对调用、上下文状态或策略绑定的任何修改, 均可通过哈希链和序列号进行密码学检测。

不可否认性 (Non-Repudiation) (实现 O5: 可问责性): 数字签名创建不可否认的证明, 将主体与其操作关联起来, 支持带有防篡改审计日志的取证分析和合规验证。

与认证系统调用 [5], [6] 类似, 核心思想在概念上很简单: 为每次实体间调用附加策略和密码学签名, 确保完整性。

Invocation = (Args, Policy, MAC)
MAC = Sign(K, Args || Policy)

其中, 消息认证码 (MAC, Message Authentication Code) 签名使用密钥或私钥 K 将参数和策略标识符进行密码学绑定。在接收端, 验证分三步进行: 验证签名以确保真实性和完整性, 检索并验证策略绑定以防止替换, 然后评估操作是否满足策略规则。

4.1. 设计原则

实现认证 workflow 需要四项原则, 这些原则组合起来提供密码学保证:

零信任身份 (Zero-Trust Identity): 每个实体——智能体、工具、数据源、LLM——拥有唯一的密码学身份 (密钥对, keypair)。即使同一进程中的实体也彼此独立验证, 不传播任何隐式信任。我们将企业身份与智能体级验证桥接: 外部主体通过企业 IAM (身份与访问管理) 认证; 运行时将这些身份传播为防篡改的会话上下文。内部使用轻量级临时签名标识智能体, 应对动态身份挑战 (第 3 节)。

边界验证 (Boundary Verification): 每次实体间通信——prompt→LLM、agent→tool、tool→data——都需要独立的密码学验证。操作携带签名和策略标识符; 接收方使用注册的公钥验证签名, 检索策略并评估约束。验证与传输协议无关——在 MCP、LangChain、OpenAI API 或自定义协议中保持一致。

策略执行点 (Policy Enforcement Points, PEP): PEP 在每个边界提供独立验证——实体不能验证自己的操作。PEP 以进程内 (链接库) 或边车 (sidecar) 方式部署, 在操作执行前验证调用, 提供水平可扩展性、零信任验证、防篡改能力和执行独立性。这依赖于 L3 (执行完整性)。

可信认证上下文 (Trustworthy Authenticated Context): 运行时提供防篡改的会话状态 (如 [3]), 确保完整性, 包括: 将操作绑定到会话范围 (上下文 ID、会话 ID)、防重放 (序列号)、篡改检测 (哈希链, hash chain) 以及 workflow 依赖 (证明前置操作已完成的密码学证言)。

协议。这些原则组合成一个四阶段协议, 每次边界穿越都是一次认证 workflow。图 2 展示了完整流程。

注册 (Registration): 每个实体注册后获得 (agent_id, keypair)。注册在最细粒度进行——智能体内的单个工具获得独立的 (tool_id, keypair), 最小化爆炸半径 (blast radius)。

调用 (Invocation): 调用方构造签名调用, 绑定操作、参数、策略和会话上下文。

验证 (Verification): 接收方 PEP 通过三个阶段独立验证: (1) 签名验证 (真实性 + 完整性), (2) 策略绑

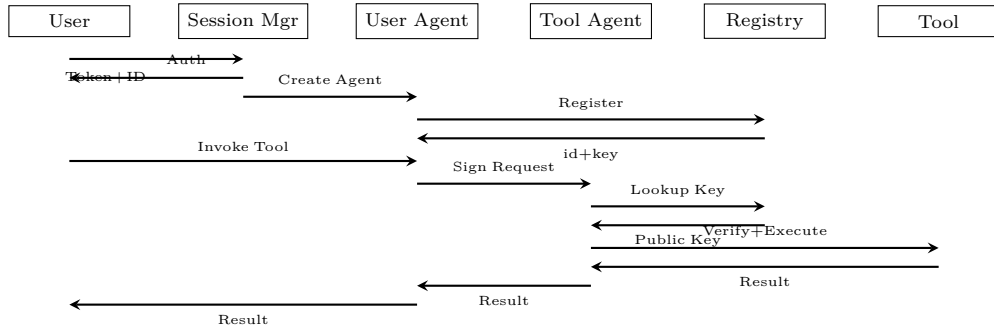


图 2. 注册与调用流程：展示认证、实体注册、签名调用及双向验证。

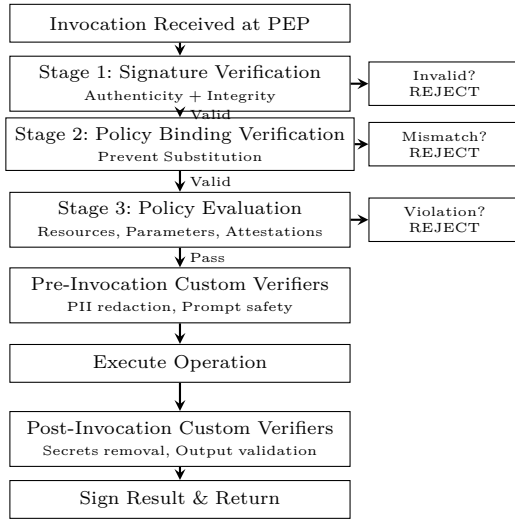


图 3. PEP 验证流程：三阶段密码学验证及可选的自定义验证器 (custom verifier)。

定验证（防止替换），(3) 策略评估（资源权限、参数约束、证言）。图 3 详细展示了验证流程。

证言 (Attestation)：操作完成后，服务对结果签名。上下文用密码学签名的证言更新，证明操作已完成，为下游操作维护认证上下文。

服务对结果签名；调用方在接受结果前验证服务签名——实现类似于双向 TLS 的双向认证 (bidirectional authentication)。该无状态协议确保每一步独立验证，无隐式信任在多个工作流步骤中传播。

4.2. 验证流程

图 3 展示了验证流水线——我们完整性主张的核心。该流水线始终强制执行且不可绕过：每次调用在执行前必须经过密码学验证，接收方使用从注册中心检索的公钥独立验证调用方。

策略构建：阶段 3 在接收方通过双视角 (dual-perspective) 执行独立构建生效策略。PEP 检索组织策略（公司、业务单元、团队）和资源特定约束，计算 $P_{\text{eff}} = \text{Intent} \cap \text{Resource}$ （第 3 节）。调用方声明意图由资源约束独立验证——调用方无法影响策略评估。交集代数（定理 1-3）保证单调约束 (monotonic restriction)：组合策略只会收窄权限，绝不会放宽。

自定义验证器 (Custom Verifiers)：MAPL 的声明式约束处理资源权限和参数，但某些安全检查需要对调用上下文执行命令式逻辑。自定义验证器扩展策略评估，包括 PII（个人身份信息）检测、SQL 注入防护、通过 LLM 分类器 (LLMs as judges) 的提示安全、速率限制和地理位置检查。调用前 (pre-invocation) 验证器在策略评估后执行；调用后 (post-invocation) 验证器处理结果。验证器在密码学验证过的上下文上运行，是由系统管理员（而非应用开发者）检入的受信代码。一旦在验证流程中声明，PEP 强制执行其运行——不可绕过。这在保持完整性保证的同时，提供了特定领域的可扩展性 (HIPAA、GDPR、SOC 2)。

四个生产级验证器展示了该架构：(1) MemoryIntegrityVerifier 通过速率限制、受保护字段强制和密码学完整性检查防止内存投毒 (OWASP LLM01)；(2) WorkflowIntegrityVerifier 通过强制前置步骤和维护证言链防止工作流劫持 (OWASP LLM04)；(3) ToolAuthorizationVerifier 通过 RBAC（基于角色的访问控制）和危险模式检测（如 `file_write`→`command_execute`）防止工具滥用 (OWASP LLM06)；(4) StorageIntegrityVerifier 通过路径遍历防护和加密强制防止数据泄露 (OWASP LLM02)。附录 A 提供了实现细节。

这种分离至关重要：核心验证流水线（阶段 1-3：签名验证、策略绑定、策略评估）提供确定性保证——通过 MAPL 声明式约束的密码学强制实现 100% 召回率且零误报（引理 1、4）。自定义验证器（阶段 4-5）是可选的行政管理扩展，可能使用启发式方法（基于 ML 的 PII 检测、LLM 安全分类器），如果配置不当可能引入误报，但不会削弱核心保证——它们只能增加限制，永不能移除。验证器仅在密码学验证通过后执行，操作对象是经认证的调用和防篡改状态，而非无界应用输入。即使验证器失败或被攻破，阶段 1-3 仍强制执行密码学完整性和策略约束。绕过验证器需要破坏受信管理员代码 (L3)，而非操纵应用逻辑 (A1)。

安全保证。面对攻击者 A1-A5（第 2 节），协议机制通过四个独立的密码学层次组合成纵深防御：**完整性**（签名建立真实性，抵御网络攻击 A4）；**授权**（策略交集强制执行意图——即使持有有效签名，操作也必须满足组合策略，防止应用控制 A1 下的权限升级）；**溯源**（哈希链和序列号使上下文篡改可检测，抵御内容注入 A2 和多轮操纵 A5）；**合规**（自定义验证器在密码学验证过的上下文上添加领域特定验证，提供不可否认性）。这些共同实现安全目标 O1-O5。

协议提供纵深防御：每层在每个边界独立运作（引理 7，第 7 节）。攻破签名验证不会绕过策略评估；篡改上下文不会击败工具 PEP；绕过某个验证器不会使密码学检查失效。第 7 节形式化了这些独立性保证。

5. 通用安全抽象

认证工作流为智能体 AI 提供通用安全抽象——我们通过保护跨三个架构层次的九个框架来验证这一主张：协议标准 (MCP、A2A)、LLM API (OpenAI、Claude) 和编排框架 (LangChain、CrewAI、AutoGen、LlamaIndex、Haystack)。无论框架架构如何，每次智能体交互都完全映射到一个可验证的边界穿越，由我们的四阶段协议控制：注册 → 调用 → 验证 → 证言。

5.1. 框架集成

表 1 展示了九个框架如何映射到认证工作流原语。以下重点介绍每个架构层的代表性示例：

协议层：MCP。MCP 服务器暴露工具、资源和提示。资源捕获旨在为 LLM 提供额外上下文的静态数据集，而 MCP 工具是具有副作用和参数的活动函数。服务器映射到我们系统中的智能体，提示映射到认证提示 (Authenticated Prompts) [3]，资源和工具抽象为受不同类型策略保护的已验证工具——每个都具有唯一独立身份和密钥对。

MCP Server → Agent with Registered Tools

Tools	Resources	Prompts
execute() params	read() watch()	getPrompt() arguments template
↓	↓	↓
Tool (w/PEP)	Tool (w/PEP) (policy-diff)	Authenticated Prompt (signed)

协议层：A2A。A2A——智能体到智能体协议——直接映射到认证工作流。智能体以密码学身份注册。委托 (delegation) 变为带有范围约束的签名令牌，遵循 MAPL 的交集代数——被委托方不能授予比接收到的更宽的权限。证言通过密码学证明实现工作流依赖，确保操作按所需顺序完成。

LLM 接口：OpenAI 和 Claude。OpenAI 和 Claude 遵循相同模式。API 端点注册为智能体。LLM 推理操作 (generate、complete、embed) 注册为带有嵌入式 PEP 的工具。两种部署变体提供了灵活性：双向包装 (two-sided wrapping) 将 LLM API 和客户端应用都视为智能体——API 侧 PEP 验证传入请求是否满足提供商策略，而客户端侧 PEP 验证函数调用是否满足应用策略；单向包装 (one-sided wrapping) 仅包装客户端工具，将 LLM API 视为透传，在信任 LLM 提供商处理提示安全且重点在于保护客户端工具调用的情况下简化部署。这解决了 LLM 不可信用于授权决策的挑战——验证必须在函数执行边界 (S2) 进行，由 PEP 独立验证签名并评估策略。

编排层：LangChain。编排框架协调跨工具和 LLM 的多步工作流，暴露多个控制点：链组合逻辑、智能体推理、内存操作、智能体间通信。仅保护单个工具将使组合和状态管理处于未保护状态。

我们的设计在四个层次处理安全。智能体以密码学身份注册。工具独立注册并带有嵌入式 PEP——每次工具调用都经过签名验证和策略评估。链 (Chains)——操作的顺序组合——通过交集强制执行策略组合 (第 3 节)；如果某个工具策略拒绝操作，将该工具组合到链中不能放宽限制——生效策略通过交集变得更加严格，永不变宽松。内存 (Memory)——跨交互的持久状态——通过认证上下文 [3] 强制上下文完整性，防止上下文投毒 (S4)。

LangChain Orchestrator (Agent with Tools)

Agents	Tools	Chains	Memory
Coordinate workflow	Individual operations	Sequential compose	Persistent state
↓	↓	↓	↓
Agent	Tool (w/PEP)	Policy ∩	Authenticated Context
Each agent has own identity + policy	Each tool verifies signed invoc. before execution	Effective policy = ∩ of all tool policies	Context integrity verified every transition

其他编排框架 (CrewAI、AutoGen、LlamaIndex、Haystack) 遵循类似模式——角色映射到证言进行密码学角色绑定，代码执行使用可插拔验证器进行 AST (抽象语法树) 分析，RAG (检索增强生成) 流水线强制执行文档访问控制，详见附录 A。

5.2. 协议级通用性

在协议层面，所有九个框架统一为同一个抽象：智能体通过签名调用发起操作；PEP 验证签名和策略；操作执行或被拒绝。这种统一性确保了异构组合中的一致安全——LangChain 编排 OpenAI 调用 MCP 工具——每个边界独立验证。

当工作流跨框架时，攻破一个框架的验证不会绕过其他框架。每个边界的 PEP 独立验证：(1) 签名有效性 (真实性)；(2) 身份 (防止冒充)；(3) 资源权限、参数约束和证言的策略评估 (授权)。这种组合方式——保护边界而非框架——实现了跨异构系统的统一保护。第 7 节通过引理 6-7 形式化了这些保证。

6. 智能体 AI 信任层

本节描述智能体 AI 信任层 (Agentic AI Trust Layer)，包含认证工作流以及 [3] 中描述的认证提示与上下文。所有九个框架通过轻量框架适配器 (framework adapter) 映射到信任层。信任层包含提供核心服务 (身份、策略、日志、路由) 的控制平面 (control plane)、与企业基础设施 (IAM、PKI、SIEM、存储等) 的集成、验证网关 (边车 PEP) 以及在智能体应用中嵌入 PEP 的客户端库。受篇幅所限，我们重点讨论关键设计要素，而非详尽概述。

6.1. 架构

图 4 展示了信任层架构，其中智能体与控制平面交互。**框架适配器 (Framework Adapters)。**每个框架通过轻量适配器集成，将框架原生操作转换为认证工作

Framework	Layer	Mapping to Authenticated Workflow Primitives
MCP	Protocol	Server→Agent, Tools→Tools (w/PEP), Resources→Tools (policy-differentiated), Prompts→AuthenticatedPrompt
A2A	Protocol	Agents→Agents (cryptographic identity), Delegation→Signed tokens (scope constraints), Attestations→Attestations (workflow dependencies)
OpenAI	LLM API	Endpoint→Agent, Operations (generate/complete/embed)→Tools (w/PEP), Two deployment variants (two-sided/one-sided wrapping)
Claude	LLM API	Endpoint→Agent, Operations (generate/complete/embed)→Tools (w/PEP), Two deployment variants (two-sided/one-sided wrapping)
LangChain	Orchestration	Agents→Agents, Tools→Tools (w/PEP), Chains→Policy intersection, Memory→AuthenticatedContext (hash chains prevent poisoning)
CrewAI	Orchestration	Crew members→Agents, Roles→Attestations (cryptographically bound), Tasks→Signed invocations (role-based authorization)
AutoGen	Orchestration	Conversation agents→Agents, Code execution→Tool (w/PEP), Policies→Pluggable verifiers (AST analysis, allowlists, sandbox constraints)
LlamaIndex	Orchestration	RAG pipeline→Agent, Query/Retrieve operations→Tools (w/PEP enforcing document access control to prevent prompt injection via retrieval)
Haystack	Orchestration	Document pipeline→Agent, Pipeline nodes→Tools (w/PEP), Sequential execution→Policy intersection (nodes cannot relax constraints)

表 1. 框架到认证 workflow 原语的映射。

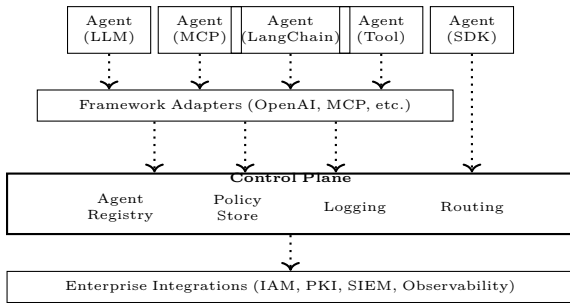


图 4. 智能体 AI 信任层。

流。适配器实现四阶段协议——注册、调用签名、验证和证言——同时呈现框架原生 API。集成是透明的；开发者使用标准框架 API (MCP 服务器、OpenAI 客户端、LangChain 智能体)，密码学强制执行自动发生。

控制平面服务。控制平面提供四项核心服务：

- **智能体注册中心 (Agent Registry)：**存储智能体标识符和公钥，以及智能体内工具的标识符和公钥，保留密码学谱系。支持即时凭证撤销。PEP 在签名验证期间检索公钥。
- **策略存储 (Policy Store)：**防篡改的策略存储，PEP 从中检索并通过交集组合层级策略（如公司、业务单元、团队）。
- **路由 (Routing)：**跨网络边界在智能体之间路由签名调用，使被调用方能够获取调用方的公钥进行验证。
- **日志 (Logging)：**通过防篡改哈希链记录所有安全事件及其密码学谱系。与企业 SIEM（安全信息与事件管理）平台集成。被阻止的操作生成结构化日志，标明拒绝原因（签名无效、策略违反、缺少证言、验证器失败），包含足够的调试细节同时避免策略泄露。审计日志支持根因分析和合规报告。

分布式执行。这些服务以高可用配置运行，与企业基础设施 (IAM、PKI、SIEM、可观测性平台) 集成。控制平面是多租户 (multi-tenant) 的——组织部署共享基础设施，同时保持租户间的密码学隔离。该架构实现零

信任分布式验证——每个 PEP 使用注册中心的公钥和策略存储中的策略独立验证调用。PEP 默认以进程内方式嵌入（链接库）或作为边车部署，提供水平可扩展性。使用嵌入式 PEP 时，不存在集中式瓶颈；验证在每个边界本地发生，开销为亚毫秒级。管理员可以通过控制平面即时撤销凭证、更新策略或隔离智能体。受篇幅所限，完整架构细节不在本文范围内；感兴趣的读者可参阅我们的技术报告获取完整实现细节。

6.2. 设计决策

在生产环境中实现认证 workflow 和信任层暴露了若干需要明确设计立场的实际挑战——我们讨论应对这些挑战的六个最有趣的设计决策。

身份桥接 (Identity Bridging)。企业身份需要跨框架桥接——每个框架有自己的认证模型 (OAuth、API 密钥、SAML)。我们不按框架逐一解决，而是在协议层进行桥接。用户认证时，信任层颁发绑定到其会话的临时密钥对 (ephemeral keypair)，并将密码学验证后的主体传播到所有框架。签名调用通过委托链携带用户身份；每个 PEP 从智能体注册中心独立验证原始主体。这集中了 IAM 集成——框架不需要各自对接企业身份系统；信任层一次性处理认证。当通过 OAuth 认证的 Alice 调用通过 API 密钥认证的 Claude，后者再调用通过 MCP 认证的工具时，日志显示“Alice via Claude accessed file”，保留审计轨迹。协议级桥接提供了更简单的编程模型和一致的主体传播，消除了按框架集成的复杂性。

会话管理 (Session Management)。许多框架缺乏原生会话概念 (MCP、LangChain 是无状态的)，但跨轮次关联对安全至关重要。我们不修改每个框架来添加会话，而是在协议层使用上下文标识符和序列号透明跟踪。信任层维护认证上下文，无论框架是否支持——没有会话的框架获得透明的协议级跟踪；有会话的框架直接映射。多租户架构确保每个会话的密码学隔离上下文，防止跨会话污染。这使得跨多框架工作流的统一审计轨迹成为可能，无需修改各框架协议。

可验证委托 (Verifiable Delegation)。多跳委托带来权限升级风险——被攻破的中间智能体可能声称拥有比授予的更大的权限。我们不依赖基于信任的委托，而是强制执行密码学范围收窄。签名委托令牌包含委托方

身份、被委托方身份、范围约束和过期时间。信任层通过交集强制执行范围收窄：委托范围 = 委托方授予 $\cap$ 被委托方拥有，扩展了 MAPL 的代数（第 3 节）。被攻破的智能体不能声称拥有比授予的更大的权限；下游服务通过密码学方式验证委托有效性。这在异构框架中提供了可证明的最小权限（least privilege）。

前向保密（Forward Secrecy）。长期 API 密钥带来操作和安全风险——密钥很少轮换，单次泄露就会危及所有会话。我们不依赖操作性的密钥轮换策略，而是在设计层面（by-design）使用临时密钥（ephemeral key）。每个会话获得唯一的临时密钥对，自动过期。密钥轮换通过会话（session）生命周期自然发生，无需基础设施修改，提供会话级前向保密。攻破一个会话的密钥不影响其他会话，在无操作开销的情况下限制了爆炸半径。

大规模服务账号。智能体系统在运行时动态生成实体——编排器（orchestrator）生成智能体，智能体生成工具，工作流创建子工作流。传统 IAM 的手动配置无法扩展到运行时动态。我们不依赖手动配置，而是在注册时（第 4 节）以工具级粒度自动配置密码学身份——分配（tool_id, keypair）并记录在智能体注册中心。无需手动配置、无需凭证分发、无 IAM 瓶颈。每个工具保持独立的密码学可问责性，最小化爆炸半径。这使得在企业规模动态部署智能体成为可能。

双向可问责性（Bidirectional Accountability）。单向认证在智能体环境中造成问责缺口——调用方无法证明服务正确执行了操作；服务无法证明调用方请求了操作。我们不使用单向签名，而是使用双向签名实现相互不可否认性。调用方对请求签名；服务对结果签名。每个操作产生来自双方的密码学证明，提供合规级审计轨迹——服务不能否认操作；调用方不能否认请求。结合身份桥接和防篡改日志，这在异构认证边界中提供了完整的可问责性。

7. 形式化分析

我们证明认证工作流在假设 L1-L3 下针对攻击者 A1-A5 实现了 O1-O5，通过围绕意图和完整性组织的七个引理。完整性属性（引理 1、2、3）确保操作是真实的且防篡改的。意图属性（引理 4、5）确保操作满足策略并安全组合。完备性属性（引理 6、7）确保所有操作都被验证，且验证是独立的。

主要安全定理。定理（认证工作流安全性）：在信任假设 L1-L3 下，认证工作流针对具有 A1-A5 能力的攻击者实现了完整性（O1）、策略执行（O2）、权限非升级（O3）、上下文完整性（O4）和可问责性（O5）。

完整性属性。这些引理确保操作是真实的、防篡改的且不可否认的——针对具有组件妥协（A3）和网络攻击（A4）能力的攻击者。

引理 1（真实性）：所有被接受的调用都具有有效的密码学签名，将主体绑定到操作、参数、策略和上下文状态。

证明：策略执行点（Policy Enforcement Point, PEP）使用从智能体注册表（第 5 节）获取的主体公钥验证签名。在密码学困难性假设 L1 下，没有私钥而伪造签名在计算上是不可行的——即使具有应用控制（A1）、网络访问（A4）或组件妥协（A3）能力的攻击者也无法为其他主体的身份生成有效签名。这实现了完整性（O1）：操作在密码学上是真实的。□

引理 2（防篡改）：对上下文状态或审计日志的修改在密码学上是可检测的，且重放攻击被阻止。

证明：上下文状态通过链接顺序状态的哈希链维护： $h_i = \text{Hash}(h_{i-1} \parallel \text{state}_i \parallel \text{sequence}_i)$ 。每个调用包含一个绑定在签名中的单调递增序列号。修改任何状态都需要找到哈希碰撞；重放旧调用会被检测到，因为 PEP 拒绝序列号小于或等于先前已处理值的调用。在密码学困难性（L1）下，找到碰撞在计算上是不可行的。多轮操纵攻击（A5）试图渐进式状态投毒或重放操作会被检测到，因为篡改会破坏哈希链或违反序列单调性。审计日志使用相同的哈希链机制。这实现了上下文完整性（O4）。□

引理 3（不可否认性）：主体不能否认执行了审计日志中记录带有有效签名的操作。

证明：只有持有私钥的主体才能生成有效签名（引理 1）。审计日志维护防篡改记录（引理 2）。因此，审计日志中的已签名操作构成了主体行为的密码学证明。这实现了可问责性（O5）。□

意图属性。这些引理确保操作满足策略并安全组合——针对具有应用控制（A1）、内容注入（A2）和渐进式攻击（A5）能力的攻击者。

引理 4（策略执行）：所有已执行操作满足由组织层级和资源约束组合而成的有效策略，且策略标识符在密码学上绑定在签名中。

证明：每个调用在签名载荷中包含一个策略标识符。PEP 检索策略并通过所有适用策略的交集计算有效策略 P_{eff} ——组织策略（基础、部门、团队）和资源特定策略（第 3 节）。试图进行策略替换攻击的攻击者（能力 A1、A4）会失败，因为修改策略标识符会使签名失效（引理 1）。策略交集确保单调约束（第 3 节，定理 1-3）——组合策略只会变得更严格，绝不会更宽松。这实现了策略执行（O2）和权限非升级（O3）。□

引理 5（组合安全）：当工作流跨越多个框架时，安全属性在框架边界之间得到保留。

证明：考虑框架 F1 上的操作 O1 调用框架 F2 上的操作 O2。两个调用都携带绑定主体和策略的签名（引理 1）。框架 F2 的 PEP 独立验证 O2 的签名，并将有效策略计算为 F1 策略和 F2 策略的交集（引理 4）。框架 F1 无法绕过 F2 的验证，因为：(a) F2 的 PEP 独立验证签名（引理 7，见下文证明），且 (b) 策略交集确保 O2 必须同时满足 F1 和 F2 的约束。跨 N 个框架的组合创建了 N 个独立验证点，组合策略为 $P_{\text{eff}} = P_1 \cap P_2 \cap \dots \cap P_N$ ，实现了单调约束。第 5 节在九个框架组合上演示了这一点。□

完备性属性。这些引理确保所有操作都被验证，且验证在各个控制面之间是独立的。

引理 6（控制面完备性与极小性）：四个控制面 {S1, S2, S3, S4} 是完备的（所有资源访问操作至少跨越一个面）且极小的（每个面都是必要的）。

通过枚举与必要性证明：

完备性：我们枚举所有资源访问操作，并展示每个操作至少跨越一个面：(1) 计算资源——LLM 推理跨越 S1（提示携带指令）；工具执行跨越 S2（特权操作）。(2) 数据资源——RAG 检索、数据库查询、网页抓取跨越 S3（外部数据流入推理）——被内容注入攻击（A2）利用。(3) 状态资源——会话状态、工作流上下文、记忆跨越 S4（跨轮次的持久状态）——被多轮操纵（A5）针对。(4) 跨智能体通信——委托跨越 S1（指令传播）或 S2（调

用)加上 S4 (上下文继承)。不访问外部资源的操作(纯计算)不需要保护——它们无法泄露数据或违反策略。

极小性:每个面都是必要的——移除任何面都会使攻击失去防护:移除 S1 (提示):通过 S3 数据 → LLM 的间接提示注入绕过验证。移除 S2 (工具):尽管有提示验证,仍可进行未授权工具执行。移除 S3 (数据):尽管有 S1/S2 验证,被投毒的 RAG 数据仍可导致工具调用。移除 S4 (上下文):跨多轮工作流的会话劫持和上下文投毒。因此,{S1, S2, S3, S4} 是完备且极小的。第 5 节在跨越三个架构层的九个框架上对此进行了实证验证。□

引理 7 (PEP 独立性):攻陷一个策略执行点 (PEP) 不会削弱其他 PEP 的验证。

证明:每个 PEP 仅使用以下要素独立验证操作:(1) 用于签名和哈希链验证的密码学原语 (L1), (2) 从可信控制面检索的公钥和策略 (L2), 以及 (3) 其自身的验证逻辑 (L3)。PEP 之间不共享运行时状态,也不进行协调。攻陷某个控制面上的 PEP (能力 A3) 的攻击者只能绕过该控制面的验证——其他控制面使用各自的 PEP 实例执行独立验证。多面攻击需要攻陷多个独立的 PEP, 每个都受 L3 保护。这实现了纵深防御 (defense-in-depth): 爆炸半径仅限于被攻陷 PEP 所在的控制面。□

主定理证明。主定理证明:根据引理 6, 所有资源访问至少跨越一个控制面。每个控制面内嵌一个 PEP 执行独立验证 (引理 1-4、7)。多框架工作流在框架边界之间组合验证 (引理 5)。验证通过策略评估执行意图 (引理 4), 通过签名 (引理 1) 和哈希链 (引理 2) 执行完整性。密码学证言 (attestation) 通过已签名、哈希链式声明提供工作流依赖关系。

每个资源访问操作有两种结果之一:(1) 授权执行——操作满足所有密码学检查 (有效签名、防篡改上下文、有效序列号) 和策略约束 (允许的资源、参数在范围内、所需的密码学证言存在), 执行后以不可否认的审计追踪记录 (引理 3); (2) 被阻止——操作至少未通过一项检查 (无效签名、策略违反、上下文篡改、序列号违规或缺失密码学证言), 在执行前被拒绝并记录违规。

具体示例:在 Q4 攻击 (第 2 节) 中,被投毒的数据指示 LLM 泄露凭证。当 LLM 生成对 `credentials.db` 的文件系统读取操作时,工具的 PEP 评估策略——该策略拒绝匹配 `*credential*` 的路径——从而阻止该操作,尽管来自已认证 LLM 的签名是有效的。策略执行防止未授权访问,无论签名是否有效。即使多个边界被攻陷,每个控制面执行独立验证 (引理 7), 限制了爆炸半径。

因此,在 L1-L3 下,具有 A1-A5 能力的攻击者无法在不受检测的情况下执行未授权资源访问,从而实现了 O1-O5。□

实践启示。这些形式化属性支撑了第 6 节中的操作性解决方案:引理 1 使能主体传播,引理 4+5 使能可验证委托,引理 2 使能会话级前向安全,引理 3 使能双向可问责性。

8. 攻击-防御验证

第 7 节证明了认证工作流在假设 L1-L3 下针对攻击者 A1-A5 实现了 O1-O5。我们通过 OWASP Top 10 覆盖 (10 项风险中覆盖 9 项)、系统化攻击测试 (11 种模式、174 个测试用例、100% 召回率、0% 误报) 以及对生产环境 CVE 的分析 (展示了在基线系统失效处提供了保护) 来验证这些形式化保证。

OWASP 大语言模型应用 Top 10 (2025) 覆盖情况。表 2 将我们的防御措施映射到 OWASP 大语言模型应用 Top 10 (2025) [7], 展示了具有明确防御机制和形式化保证的系统化覆盖。

这些防御措施应对所有攻击者能力 (第 2 节): 策略层面 (by-policy) 机制通过运行时执行阻止 A1、A2、A5 (引理 4、5); 设计层面 (by-design) 机制通过密码学困难性消除 A3、A4 (引理 1、2、3)。两者结合, 实现了 O1-O5。这验证了双重框架: 单独任一机制都不足够; 两者结合才能提供完整保护。

系统化攻击验证。在 OWASP 的宽泛风险类别之外, 我们分析了攻击机制——即攻击如何在组合工作流中体现。我们识别出 8 个机制类别, 涵盖提示操纵、工具链式调用、凭证窃取、数据泄露、恶意软件/代码执行、资源耗尽、多智能体攻击和策略违反。详细分类见附录 C。表 3 展示了 11 种攻击模式, 提供了代表性覆盖。

每种模式包含多个测试场景: 数据泄露测试批量导出、多资源访问、PII 提取、未授权访问; 推断攻击测试属性推断、成员推断、重建攻击。我们的 174 个测试用例涵盖框架组合 (OpenAI、Anthropic、LangChain 等) 和配置变体。

表 3 展示了防御分类。设计层面 (by-design) 机制实现了完整性属性 (引理 1、2、3), 使攻击在密码学上不可能: 身份伪造、会话重放、策略替换、上下文篡改、审计操纵、证言伪造。策略层面 (by-policy) 机制实现了意图属性 (引理 4、5), 通过运行时验证阻止违规: 提示注入、权限提升、凭证泄露、数据收割、供应链、资源耗尽、跨智能体攻击。复杂攻击 (如供应链攻击) 需要两者兼具——代码签名加工具审批策略。六个类别有显式实现; 多智能体攻击通过独立 PEP 验证 (引理 7) 被阻止。

实证结果: 我们的评估在 174 个测试用例上实现了 100% 召回率 (recall) 且零误报 (false positive)。设计层面 (by-design) 的消除提供了确定性保证——违规是通过密码学方式检测的, 而非模式匹配。突破验证需要解决计算困难问题 (L1), 而非精心构造对抗性输入, 这解释了为什么绕过语义防御的攻击 (Atlas、GitHub MCP CVE) 被认证工作流确定性阻止。

真实世界攻击验证。我们针对两个已攻陷生产系统的攻击验证了认证工作流——OpenAI 的 Atlas 智能体浏览器和 GitHub 的模型上下文协议 (Model Context Protocol, MCP) 服务器。这两个攻击都成功突破了使用语义护栏的基线防御; 但两者都被认证工作流通过控制面的独立验证完全阻止。随后我们通过企业集成场景展示了部署可行性。

OpenAI Atlas 浏览器攻击。OpenAI 的 Atlas 智能体浏览器 [1] 通过提示注入 (prompt injection) 导致凭证泄露 (exfiltration) 而被攻陷 (OWASP LLM01、LLM06、LLM02)。

我们模拟了该攻击: 一个合法工作流读取财务报告; 恶意输入嵌入隐藏指令以泄露凭证。该攻击成功突破了语义护栏, 因为护栏无法区分对抗性提示和合法文档内容。

策略执行阻止了这一连锁反应。文件系统策略拒绝凭证路径; 网络策略限制外部端点。PEP 验证 (引理 4) 在执行前拒绝未授权操作, 无论 LLM 的推理结果如何。我们在不同框架变体上测试了白名单、黑名单及其组合。全部阻止了攻击且零误报。

表 2. OWASP 大语言模型应用 Top 10 (2025) 覆盖情况

OWASP 风险	我们的攻击测试	防御机制	对应节
LLM01: Prompt Injection	Prompt Injection, Atlas	By-policy: Policy blocks unauthorized operations + MemoryIntegrityVerifier prevents goal manipulation	\$4, \$7
LLM02: Info Disclosure	Token Hijack, Data Exfil, Side Channel, Inference	By-policy: Resource denial + StorageIntegrityVerifier detects exfiltration patterns	\$4, \$5
LLM03: Supply Chain	Rogue Tool, Supply Chain	By-design + By-policy: Code signing + tool registry with approval policies	\$5, \$7
LLM04: Data Poisoning	Context tamper	By-design: Hash chains + WorkflowIntegrityVerifier enforces step sequences	\$4, \$7
LLM05: Output Handling	(Cross-cutting)	By-policy: Post-invocation verifiers sanitize outputs	\$5
LLM06: Excessive Agency	Keys to Kingdom, Confused Deputy	By-policy: Policy composition + ToolAuthorizationVerifier enforces RBAC with pattern detection	\$3, \$7
LLM07: Prompt Leakage	Prompt variants	By-design: Cryptographic segmentation (Auth-Prompt)	\$4
LLM08: Vector Weak.	(RAG scenarios)	By-policy: Attestations verify sources; policies restrict access	\$3, \$5
LLM09: Misinform.	(Tangential)	Pluggable verifiers; not primary security focus	\$5
LLM10: Unbounded Cons.	Denial of Service	By-policy: Rate limiting + resource quotas	\$5

覆盖情况：10 项 OWASP 风险中覆盖 9 项。LLM04 部分覆盖（运行时投毒已阻止；训练数据超出范围）。LLM09 间接相关（质量 vs. 安全）。

表 3. 已验证的攻击模式与防御分类

攻击模式	类别	防御类别	OWASP
Prompt Injection	Prompt Manipulation	By-Policy	LLM01
Keys to Kingdom	Tool Chaining	By-Policy	LLM06
Confused Deputy	Tool Chaining	By-Policy	LLM06
Token Hijacking	Credential Theft	By-Policy	LLM02, LLM06
Session Fixation	Credential Theft	By-Design	Beyond OWASP
Data Exfiltration	Data Exfiltration	By-Policy	LLM02
Side Channel	Data Exfiltration	By-Policy	LLM02
Inference Attack	Data Exfiltration	By-Policy	LLM02
Rogue Tool	Malware/Code Exec	By-Policy	LLM03
Supply Chain	Malware/Code Exec	By-Policy	LLM03
Denial of Service	Resource Exhaustion	By-Policy	LLM10
总计：11 种模式，18 个显式变体，174 个测试用例（含框架排列组合）			

覆盖情况：8 个机制类别中 6 个显式实现；多智能体攻击和策略违反攻击通过组合机制处理（见正文）。

GitHub MCP 提示注入。 GitHub 的 MCP 服务器 [8] (558,846+ 下载量) 通过 GitHub Issue 中的恶意提示调用 MCP 文件系统工具进行凭证泄露而被攻陷 (OWASP LLM01、LLM03、LLM02)。

多重屏障阻止了这一连锁反应。首先，工具授权要求来自已批准注册表的密码学签名——恶意工具在发现阶段即被拒绝（引理 1）。其次，文件系统策略阻止凭证访问（引理 4）。第三，跨框架组合确保文件系统 PEP 独立验证操作（引理 5）。该攻击通过多个控制面上的独立验证被阻止（引理 7）。

这两个攻击都表明仅靠应用层防御是不够的——尽管有语义护栏，生产系统仍被攻陷。认证工作流程通过纵深防御 (defense-in-depth) 实现完整保护：多个独立的密码学屏障必须同时被突破。所有测试场景保持了 100% 召回率且零误报，验证了形式化保证（第 7 节）可以转化为生产部署。第 10 节将这些结果与现有防御进行了对比。

9. 性能

我们通过使用 ECDSA-256 签名和 SHA-256 哈希在通用硬件（8 核、16GB RAM）上进行微基准测试 (microbenchmark) 来验证实际开销。**密码学操作**通常开销低于 ~0.2ms (ECDSA 签名生成 ~102μs, 验证 ~89μs, 每次操作哈希链更新 <15μs)。**策略操作**：检索延迟取决于后端：内存 (~5-50μs)、文件系统 (~30-200μs)、Redis

(~50-300μs)；策略交集取决于组合方式——典型 3-5 层深度的层级结构下保持在 100μs 以内。**自定义验证器**：确定性检查 (PII 检测、路径验证) 增加 30-400μs；基于 LLM 的验证器（提示安全性）增加 150-500ms，启用时占主导地位，但提供与密码学保证正交的语义分析。**端到端影响**：对于网络绑定操作 (LLM 推理、远程 API)，密码学开销 (~0.2ms) 相对于网络延迟 (50-500ms) 可以忽略不计。对于本地工具调用，不含 LLM 验证器时，PEP 验证总开销保持在 1ms 以下。审计日志通过缓冲 I/O 异步 (asynchronous) 进行。

配套论文提供了全面的系统评估，包括生产工作负载特征化、跨越全部九个框架的框架集成成本、大规模部署经验以及基于生产级基准测试的详细性能分析。

10. 相关工作

最近的综述 [9]–[12] 指出智能体 (agentic) AI 安全的四个关键缺口：不可预测的多步工作流、跨框架组合、可变环境以及不可信实体交互。这些综述诊断了碎片化问题，但缺乏系统性解决方案。先前工作要么关注应用层语义，要么关注基础设施原语；我们提供一个位于两者之间的信任层 (trust layer)——协议级原语保护四个基本控制面 (control surface)（提示、工具、数据、上下文），所有智能体与资源的交互均在此发生。

应用层防御在组合场景下失效。 框架特定的安全机制各自孤立运行。LangChain 提供模式定义与验证 [13]；

MCP 实现认证与权限控制 [14]; OpenAI 实施速率限制与内容审核 [15]。近期的漏洞暴露了组合层面的缺口: CVE-2024-8309 (LangChain 提示注入 (prompt injection) → SQL, CVSS 9.0) [16]、CVE-2024-36480 (通过未净化的 eval 实现远程代码执行) [17]、CVE-2025-6514 (MCP 供应链 (supply chain), 558K+ 下载量) [8]、Atlas 浏览器攻击 (2025 年 10 月) [1], [2]。框架边界之间不存在密码学绑定。

近期的学术工作涵盖了提示注入防御 [18]–[20]、工具使用安全 [21] 以及 RAG (检索增强生成) 检索投毒 [22], [23]——恶意指令嵌入文档中, 破坏推理过程。语义 (semantic) 防御 (护栏 (guardrail) [24]–[26]) 实现了 60–80% 的检测率, 但仍然可被绕过 [27], [28]。我们通过纵深防御 (defense in depth) 提供确定性 (deterministic) 保证 (100% 召回率, 零假阳性): 设计即消除 (by-design elimination) 使攻击在密码学上不可行; 策略即执行 (by-policy enforcement) 阻断违规。两种方法是互补的——语义防御缩小攻击面; 密码学强制执行提供不可绕过的验证。

基础设施原语缺乏工作流语义。基础设施原语提供强保证, 但缺乏工作流语义。身份系统 (SPIFFE [29]、AWS IAM (身份与访问管理) [30]) 和策略引擎 (OPA [31]、AWS Cedar [32]) 通过交集组合策略, 但无法表达时序依赖——“操作 B 需要证明 A 已完成”。Cedar 通过交集组合策略, 但缺乏: (1) 基于密码学证言 (attestation) 的工作流依赖, (2) 运行时解析的动态主体 (principal), (3) 跨四个控制面的密码学绑定。没有先决操作的密码学证明, 策略只能信任应用层报告的状态, 而攻击者可以伪造这些状态。

MAPL 通过密码学证言解决这一问题——不可伪造的密码学证明表明操作已完成, 从而实现可证明正确的多步工作流。与平台证言 (TPM [33]、SGX [34]) 通过度量证明代码完整性不同, MAPL 证言证明的是工作流完成——即操作已使用特定结果执行的密码学证明, 从而实现了基于度量的证言所不可能的时序依赖。我们将认证系统调用 [5] 和行内引用监控器 [6], [35] 从内核边界扩展到四个控制面, 通过认证上下文为工作流依赖添加有状态验证, 并通过可插拔验证器 (verifier) 支持领域特定检查 (附录 A)。

正交方法。AI 安全 [36]–[38] 作用于训练阶段; 我们提供运行时强制执行。机密计算 [34], [39] 保护数据机密性; 我们关注工作流完整性。这些方法是互补的。

11. 结论

我们提出了认证工作流 (authenticated workflows)——面向企业级智能体 AI 的首个完整信任层。我们的关键架构洞察是: 定位在协议层——介于在组合场景下失效的应用层防御与缺乏工作流语义的基础设施原语之间——能够在对异构框架提供统一保护的同时, 保持工作流上下文。所有智能体与资源的交互均经过四个基本控制面 (提示、工具、数据、上下文); 统一保护这些控制面, 实现了在任一相邻层单独无法达成的组合安全性。

三项贡献实现了这一愿景: MAPL 通过密码学证言提供密码学工作流依赖, 从而在无需信任应用层报告状态的前提下实现顺序约束; 嵌入式独立 PEP (策略执行点) 实现纵深防御, 攻击者必须同时攻破多个密码学屏障; 通过薄适配器 (adapter) 集成九个框架, 验证了控制面的完备性。形式化证明 (引理 1–7、定理 1–3) 建立了

安全属性; 实证验证在 174 个测试用例上展示了 100% 召回率、零假阳性, 以及对两个攻破基线系统的生产环境 CVE 的防护。这解决了第 1 节提出的五项挑战: 跨框架的密码学完整性、规模化策略执行、组合过程中的权限不提升、通过哈希链 (hash chain) 实现的上下文完整性、以及通过不可否认性 (non-repudiation) 实现的完全可问责性。随着智能体 AI 向生产环境过渡, 认证工作流提供了安全基底, 使得在组合缺口当前阻碍采用的场景下能够安全部署。

致谢

感谢匿名审稿人提供的宝贵反馈。本研究部分受 [盲审隐去] 资助。

参考文献

- [1] Security Researchers, “Prompt injection vulnerability in OpenAI Atlas browser,” Security Advisory, October 2025, disclosed October 21, 2025.
- [2] OpenAI, “Statement on prompt injection challenges from OpenAI CISO,” Official Statement, October 2025.
- [3] M. Rajagopalan and V. Rao, “Protecting context and prompts: Deterministic security for non-deterministic AI,” in *Proceedings of the 1st International Workshop on AI Security and Evaluation (IASAI’26)*, co-located with *IEEE Symposium on Security and Privacy*. MACAW Security, Inc. and ROOST.tools, 2026, to appear.
- [4] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for mind exploration of large scale language model society,” *arXiv preprint arXiv:2303.17760*, 2023.
- [5] M. Rajagopalan, M. A. Hiltunen, T. Jim, and R. D. Schlichting, “Efficient and flexible security support for distributed component-based systems,” *IEEE Transactions on Dependable and Secure Computing*, vol. 3, no. 4, pp. 352–369, 2006.
- [6] Ú. Erlingsson and F. B. Schneider, “The inlined reference monitor approach to security policy enforcement,” *ACM Transactions on Programming Languages and Systems*, vol. 28, no. 1, pp. 2–44, 2004.
- [7] OWASP Foundation, “OWASP top 10 for LLM applications 2025,” <https://genai.owasp.org/>, 2025.
- [8] Invariant Labs, “CVE-2025-6514: Prompt injection in GitHub model context protocol server,” <https://www.invariant.sh/blog/cve-2025-6514>, May 2025, disclosed May 26, 2025; 558,846+ downloads.
- [9] Y. Wang, X. Chen, J. Li, and W. Zhang, “AI agents under threat: A survey of key security challenges and future pathways,” *ACM Computing Surveys*, 2024, arXiv:2406.02630.
- [10] Y. Zhou, T. Liu, and J. Wang, “A survey on agentic security: Applications, threats and defenses,” *arXiv preprint arXiv:2510.06445*, 2025.
- [11] N. Gong, Y. Liu, and Y. Zhang, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proceedings of the 33rd USENIX Security Symposium*. USENIX, 2024.
- [12] Y. Liu, X. Chen, and Z. Wang, “From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows,” *arXiv preprint arXiv:2506.23260*, June 2025.
- [13] L. Inc., “Langchain: Building applications with llms,” <https://www.langchain.com/>, 2023.
- [14] Anthropic, “Model context protocol (MCP),” <https://modelcontextprotocol.io/>, 2024.

[15] OpenAI, “OpenAI Assistants API,” <https://platform.openai.com/docs/assistants/>, 2024.

[16] Huntr Dev, “CVE-2024-8309: Prompt injection in LangChain’s GraphCypherQACHain leads to full database compromise,” <https://nvd.nist.gov/vuln/detail/CVE-2024-8309>, 2024.

[17] NVD, “CVE-2024-36480: Remote code execution in LangChain,” <https://nvd.nist.gov/vuln/detail/CVE-2024-36480>, 2024, cVSS 9.0 - Critical.

[18] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” *arXiv preprint arXiv:2211.09527*, 2022.

[19] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023, pp. 1–15.

[20] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, “Baseline defenses for adversarial attacks against aligned language models,” *arXiv preprint arXiv:2309.00614*, 2023.

[21] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” *arXiv preprint arXiv:2309.15817*, 2023.

[22] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge poisoning attacks to retrieval-augmented generation of large language models,” in *arXiv preprint arXiv:2402.07867*, 2024.

[23] Z. Cheng, G. Li, J. Liu, Q. Zhang, and J. Shao, “PARDEN: Toward detecting backdoor attacks in retrieval-augmented generation,” *arXiv preprint arXiv:2409.01573*, 2024.

[24] NVIDIA, “NeMo Guardrails: Programmable guardrails for LLM applications,” <https://github.com/NVIDIA/NeMo-Guardrails>, 2023.

[25] Rebuff.ai, “Rebuff: Prompt injection detection,” <https://github.com/protectai/rebuff>, 2023.

[26] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, “Llama Guard: LLM-based input-output safeguard for human-ai conversations,” 2023.

[27] A. Wei, N. Haghtalab, and J. Steinhardt, “Jailbroken: How does LLM safety training fail?” *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[28] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.

[29] CNCF, “SPIFFE: Secure production identity framework for everyone,” <https://spiffe.io/>, 2023.

[30] Amazon Web Services, “AWS identity and access management: Policy composition and authorization,” <https://aws.amazon.com/iam/>, 2023.

[31] CNCF, “Open policy agent: Policy-based control,” <https://www.openpolicyagent.org/>, 2023.

[32] A. W. Services, “Cedar: Policy language for authorization,” <https://www.cedarpolicy.com/>, 2023.

[33] Trusted Computing Group, “TPM 2.0 library specification,” Trusted Computing Group, Tech. Rep., 2016, specification Family “2.0”, Level 00 Revision 01.38.

[34] V. Costan and S. Devadas, “Intel SGX explained,” *IACR Cryptology ePrint Archive*, vol. 2016, p. 086, 2016.

[35] F. B. Schneider, “Enforceable security policies,” *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[36] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon *et al.*, “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.

[37] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.

[38] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse *et al.*, “Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned,” *arXiv preprint arXiv:2209.07858*, 2022.

[39] C. Gentry, “A fully homomorphic encryption scheme,” Ph.D. dissertation, Stanford University, 2009.

Appendix

附录 A: MAPL 策略示例

基础组织策略

```
{
  "policy_id": "acme:base",
  "resources": ["tool:*", "
    ↪ llm:*"],
  "denied_resources": [{"*
    ↪ credential*"}],
  "constraints": {
    "attestations": [{"
      ↪ user_authenticated
      ↪ "}]}
}
```

基线策略，包含凭证阻断。

密码学证言链式传递

```
// Step 1: Analysis
{"policy_id": "fin:analyze
  ↪ ",
  "resources": [{"tool:
    ↪ analyze"},
  "constraints": {"
    ↪ attestations":
    [{"user_authenticated"}]}
// -> Produces:
  ↪ analysis_done
```

```
// Step 2: Requires Step 1
{"policy_id": "fin:report",
  "resources": [{"tool:report
    ↪ "}],
  "constraints": {"
    ↪ attestations":
    [{"analysis_done"}]}
// -> Produces: report_done
```

```
// Step 3: Requires Step 2
{"policy_id": "fin:send",
  "resources": [{"tool:email
    ↪ "}],
  "constraints": {"
    ↪ attestations":
    [{"report_done"}]}

```

密码学证明防止 workflow 劫持。

部门策略（财务）

```
{
  "policy_id": "acme:finance
    ↪ ",
  "extends": "acme:base",
  "resources": [{"data:
    ↪ finance:*"}],
  "constraints": {
    "attestations": [{"
      ↪ mfa_verified"}]}
}
```

继承基线策略，增加多因子认证。

资源策略（数据库）

```
{
  "policy_id": "database:
    ↪ customer_db",
  "constraints": {
    "denied_parameters": {
      "query": [{"DROP", "
        ↪ DELETE"}]}
}
```

双视角约束。

委托（Delegation）策略

```
{
  "policy_id": "agent:
    ↳ sub_agent",
  "extends": "acme:finance",
  "resources": [{"data":
    ↳ finance:reports
    ↳ :*}],
  "constraints": {
    "parameters": {
      ↳ date_range":
        ["2024-Q1", "2024-Q2"]}
}
```

狭窄的委托范围。

应急访问

```
{
  "policy_id": "emergency:
    ↳ incident",
  "extends": "acme:base",
  "validity": {
    "not_after": "2024-10-25
      ↳ T16:00:00Z"
  },
  "constraints": {
    "attestations":
      ["ciso_approved"]}
}
```

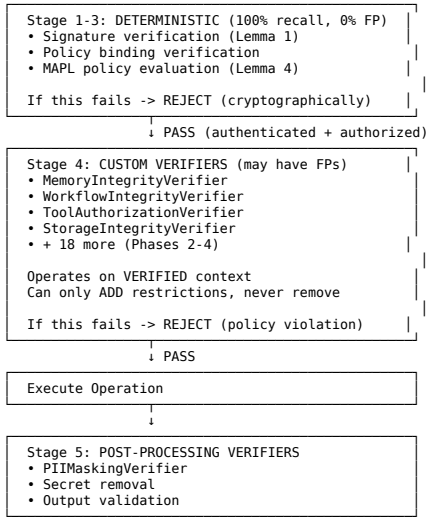
时间限定（2 小时窗口）。

附录 B：自定义验证器实现

验证流水线架构

PEP 验证流水线由五个阶段组成：

Invocation Arrives at PEP



即使自定义验证器被攻破，阶段 1–3 仍强制执行密码学完整性和策略约束。

生产环境验证器

四个生产环境验证器通过在密码学验证过的上下文上执行领域特定检查来应对 OWASP Top 10 威胁：

MemoryIntegrityVerifier
防止内存投毒（OWASP LLM01）

- 速率限制（10 更新/分钟）
- 保护字段（goals, system_prompt）
- 模式检测（可疑更新）
- 密码学完整性检查

ToolAuthorizationVerifier
防止工具滥用（OWASP LLM06）

- RBAC（角色到工具的映射）
- 危险模式（write→execute）
- 使用速率限制（30 工具/分钟）
- 权限提升防护

WorkflowIntegrityVerifier
防止工作流程劫持（OWASP LLM04）

- 顺序执行（前置条件）
- 状态跟踪（防止重复执行）
- 证言链（密码学证明）
- 持续时间限制（最大 3600 秒）

StorageIntegrityVerifier
防止数据泄露（OWASP LLM02）

- 路径遍历防护
- 数据分类（secret/sensitive）
- 加密强制执行
- 泄露检测（批量读取）

附录 C：框架集成详情

详细映射，展示九个框架的框架构造如何映射到认证 workflow 原语。

A2A（智能体间协议）

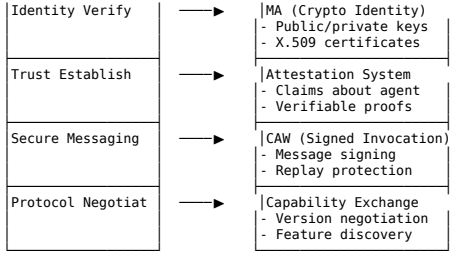
A2A 定义了智能体发现、信任建立和通信的标准。MACAW 为 A2A 提供所需的密码学基础设施：

A2A Requirements:

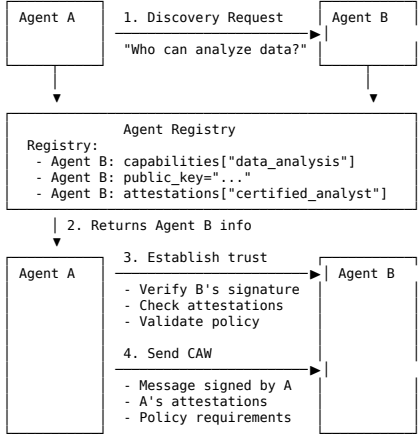
Agent Discovery

MACAW Already Provides:

Agent Registry - Agent directory - Capability ads



Complete A2A Flow:



安全挑战：智能体在委托权限之前必须验证身份和能力。密码学证言提供密码学声明，使得无需中心化权威机构即可做出信任决策。

MCP（模型上下文协议）

MCP 服务器暴露三种能力类型：工具（可执行函数）、资源（带读/写/监听操作的数据访问）和提示（可检索模板）。MACAW 映射如下：

MCP Core Concepts:

MCP Server		
Tools execute()	Resources read/write()	Prompts getPrompt()

MACAW Mapping:

macawAgent (MCP Server)		
Tool (execute → CAW)	Tool (Resource) (read → CAW)	Authenticated Prompt (signed template)

Resource Flow:

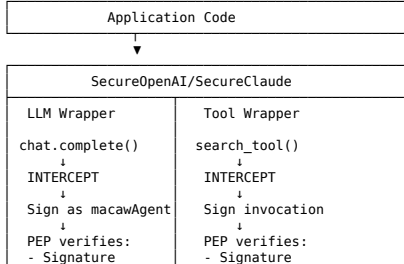
Client: "Read file.txt" → SecureMCP: invoke_tool()
→ PEP: signature valid? → Policy: allow read?
→ Execute if allowed

安全挑战：资源代表只读数据访问，而工具可以修改状态。该映射通过策略强制执行只读约束，防止从数据检索到数据修改的权限提升（privilege escalation）。

OpenAI 与 Claude（LLM 接口）

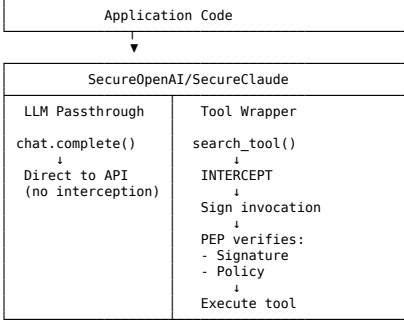
LLM 提供商暴露带函数调用的推理 API。基于信任假设，存在两种部署变体：

Two-sided Wrapping: Both LLM API and Client as macawAgents



- Provider policy ↓ Execute LLM call	- App policy ↓ Execute tool
--	-----------------------------------

One-sided Wrapping: Only Client Tools as macawAgents



安全挑战：LLM 不可信任用于授权决策。即便使用提示工程，对抗性提示仍可能导致未授权的函数调用。PEP 在函数执行边界进行验证，不依赖于 LLM 的决策。

LangChain（多智能体编排）

LangChain 协调跨工具和 LLM 的多步工作流。多层集成涵盖四个层面：

LangChain Architecture:

LangChain Orchestrator (macawAgent)			
Agents	Tools	Chains	Memory
Coordinate workflow	Individual operations	Sequential compose	Persistent state across turns
Agent → macawAgent	Tool → Tool (w/PEP)	Chain → Policy Intersect	Memory → AuthenticatedCntx (hash chains)
Each agent has own identity and policy	Each tool verifies CAW before execution	Effective policy = $\cap$ of all tool policies	Context integrity verified every state transition Prevents context poisoning attacks

Workflow Flow:

User: "Analyze documents and send summary"

LangChain Agent (macawAgent)

Step 1: Read docs

→ Tool invocation → CAW → PEP check → Execute

Step 2: Analyze content

→ LLM invocation → CAW → PEP check → Execute

Step 3: Send email

→ Tool invocation → CAW → PEP check → Execute

Policy Composition via Intersection:

If tool_1 policy allows {A, B, C}

And tool_2 policy allows {B, C, D}

And chain composes tool_1 → tool_2

Then effective policy = {B, C} (intersection)

Chain composition cannot relax tool policies.

安全挑战：编排器（orchestrator）暴露了超出单个工具的多个控制点——链组合逻辑、智能体推理、内存操作。仅保护工具会使组合和状态管理失去防护。

CrewAI（基于角色的多智能体）

CrewAI 将智能体组织为 crew，每个成员具有角色并以协作方式执行任务：

CrewAI Architecture:

Crew (macawAgent)			
Members	Roles	Tasks	Collaboration
Agents with specific roles	Role-based perms	Work items assigned to agents	Inter-agent communication
Member → macawAgent	Role → Attestation	Task → CAW	Message → CAW with role check

Writer cannot access DB tools	"writer" attested crypto-graphical	Execute task only if role matches	Writer cannot invoke DB tools reserved for Researcher role
-------------------------------	------------------------------------	-----------------------------------	--

Role-Based Flow:

Crew: Researcher + Writer

Task: "Write report on Q4 data"

Researcher (macawAgent with "researcher" attestation):

→ Invokes database_query tool

→ PEP checks: signature valid? role="researcher"?

→ Policy: "database_query" requires attestation["researcher"]

→ Allowed: execute query

Writer (macawAgent with "writer" attestation):

→ Receives data from Researcher

→ Attempts database_query tool

→ PEP checks: signature valid? role="writer"?

→ Policy: "database_query" requires attestation["researcher"]

→ Denied: Writer lacks required attestation

安全挑战：基于角色的权限必须通过密码学强制执行。“writer”角色不应访问为“researcher”保留的数据库，即使驱动 writer 的 LLM 尝试未授权调用。密码学证言将角色以密码学方式绑定到智能体身份。

AutoGen（自主代码生成）

AutoGen 支持自主代码生成和执行。安全挑战在于无限制的代码执行：

AutoGen Architecture:

AutoGen Conversation (macawAgent)			
Agents	Code Gen	Exec	Validation
Conversational agents	Generate Python code to solve tasks	Execute generated code	Verify code before execution
Agent → macawAgent	LLM → Tool	Exec → Tool (w/PEP)	Verifiers: - AST analysis - AllowList check - Sandbox verify
Each agent signs requests	Code gen becomes CAW with code	Execution request = CAW	Pre-invocation verifiers analyze code content before execution

Code Execution Flow:

User: "Calculate Fibonacci(10)"

AutoGen Agent generates code:

def fib(n): return n if n<2 else fib(n-1)+fib(n-2)

Execution request → CAW

PEP invokes pre-execution verifiers:

1. AST analysis: detect dangerous patterns (os.system, subprocess, eval, exec, import sys)
2. AllowList check: only math/basic operations
3. Sandbox verification: no network/file access

Verifiers approve → Execute in sandbox → Return result

Malicious attempt: "os.system('rm -rf /')"

AST analysis detects os.system → DENY

安全挑战：无限制的代码执行允许任意操作，包括数据泄露（exfiltration）和权限提升。可插拔验证器在执行前通过 AST 解析分析代码，强制执行白名单和沙箱（sandbox）约束。

LlamaIndex（RAG 流水线）

LlamaIndex 提供 RAG（检索增强生成）流水线，用于文档处理和问答：

LlamaIndex Architecture:

LlamaIndex Pipeline (macawAgent)			
Index	Query Engine	Retrieve	Generate
Document indexing	Process queries	Vector search	LLM generates answer from retrieved docs
Index → macawAgent	Query → Tool (w/PEP)	Retrieve → Tool (w/PEP)	Generate → Tool (w/PEP)
Vector DB becomes macawAgent	Query validated against policy	Document access validated against	LLM receives only allowed documents

		policy	
--	--	--------	--

```
RAG Flow with Security:
User: "What were Q4 earnings?"
↓
Query Engine (Tool with PEP)
→ CAW: query="Q4 earnings"
→ PEP checks:
  - Signature valid?
  - Policy allows query pattern?
  - Denied patterns: *password*, *credentials*
→ Approved: continue
↓
Retriever (Tool with PEP)
→ CAW: retrieve docs matching "Q4 earnings"
→ PEP checks:
  - Can access these documents?
  - Allowed files: *public*, *reports*
  - Denied files: *private*, *secret*
→ Approved: retrieve documents
↓
Generator (Tool with PEP)
→ CAW: generate answer from retrieved docs
→ PEP checks: output validation
→ Return answer to user
```

安全挑战：RAG 流水线访问不可信的文档存储。恶意文档可能包含提示注入或敏感数据。PEP 在检索前验证文档访问模式和查询约束。

Haystack（文档处理流水线）

Haystack 提供文档处理流水线，包含用于检索、预处理和生成的节点：

Haystack Architecture:

Haystack Pipeline (macawAgent)			
Pipeline	Nodes	Retrieve	Process
Sequential execution of nodes	Pipeline steps	Document retrieval	Text processing and generation
Pipeline → macawAgent	Node → Tool (w/PEP)	Retrieve → Tool (w/PEP)	Process → Tool (w/PEP)
Each node in chain enforces policy	Each node verifies CAW	Document access controlled by policy	Processing validated against policy

```
Pipeline Flow:
User: "Process documents in ./reports/"
↓
Pipeline (macawAgent)
↓
Node 1: FileReader (Tool with PEP)
→ CAW: read("./reports/")
→ PEP checks:
  - Allowed paths: *reports*, *public*
  - Denied paths: *credentials*, *private*
→ Approved: read files
↓
Node 2: Preprocessor (Tool with PEP)
→ CAW: clean text, remove PII
→ PEP checks: PII detection verifier
→ Execute with redaction
↓
Node 3: Generator (Tool with PEP)
→ CAW: generate summary
→ PEP checks: output validation
→ Return result

Policy Composition Across Nodes:
If Node 1 allows {A, B, C}
And Node 2 allows {B, C, D}
And pipeline chains Node 1 → Node 2
Then effective policy = {B, C} (intersection)
```

安全挑战：文档处理流水线访问文件系统和外部数据源。节点必须强制执行路径约束和内容验证，以防止未授权访问或数据泄露。

附录 D：AI 智能体威胁分类法

ID	攻击	描述与影响
类别 A：提示注入与输入操控		
T-A1	隐藏提示注入	通过白底白字、CSS、HTML 注释嵌入。影响：命令执行。
T-A2	语义混淆	重新措辞以逃避过滤器，base64 编码。影响：过滤器绕过。
T-A3	多轮投毒	跨轮次渐进注入，伪造历史记录。影响：权限提升。
T-A4	延迟激活	条件触发器（定时炸弹）。影响：逃避即时检测。
类别 B：工具链式调用与派生利用		
T-B1	链式权限提升	组合良性操作（search→list→read）。影响：策略绕过。
T-B2	语义漂移	意图渐进偏离。影响：复合解释错误。
T-B3	派生深度耗尽	深度递归链。影响：资源耗尽，策略漂移。
类别 C：凭证与会话攻击		
T-C1	令牌提取	从上下文中读取凭证。影响：会话劫持，云访问。
T-C2	凭证泄露	工具滥用序列（read→email）。影响：持久化未授权访问。
T-C3	OAuth 流程操控	将 OAuth 诱导至攻击者应用。影响：账户接管。
T-C4	会话重放	复用有效的签名提示。影响：过期认证利用。
类别 D：数据泄露与隐私侵犯		
T-D1	对话历史挖掘	从历史记录中提取 PII（个人身份信息）/密钥。影响：G
T-D2	跨主体泄露	上下文隔离不足。影响：未授权跨用户访问。
T-D3	嵌入空间投毒	污染向量数据库/RAG。影响：持久数据污染。
T-D4	文档批量窃取	系统化文件提取。影响：大规模数据盗窃。
类别 E：恶意软件与代码执行		
T-E1	AI 调用的恶意软件	下载/执行恶意代码。影响：勒索软件，后门。
T-E2	代码生成利用	生成伪装的恶意脚本。影响：资源劫持。
T-E3	持久化安装	创建后门（.bashrc, cron）。影响：持久化入侵。
T-E4	软件包供应链攻击	安装被木马化的依赖。影响：依赖混淆。
类别 F：资源耗尽与成本攻击		
T-F1	计算耗尽	递归操作，无限循环。影响：拒绝服务，账单膨胀。
T-F2	API 速率限制耗尽	循环耗尽配额。影响：服务降级，数千美元以上成本。
T-F3	存储/带宽饱和	无限制下载/生成。影响：系统故障。
类别 G：多智能体系统攻击		
T-G1	智能体间感染	通过 MCP/A2A 投毒提示。影响：级联入侵。
T-G2	工作流劫持	构造跨智能体请求。影响：分布式策略绕过。
T-G3	拜占庭式智能体行为	持有有效凭证的恶意智能体。影响：授权攻击。
T-G4	信任锚点破坏	破坏注册中心/身份提供方。影响：完全入侵。
类别 H：策略与合规违规		
T-H1	策略混淆	利用规则歧义。影响：授权缺口。
T-H2	审计追踪篡改	破坏日志，虚假条目。影响：取证证据丢失。
T-H3	合规绕过	GDPR/HIPAA/SOC 2 违规。影响：法律责任，4% 营
T-H4	证言伪造	伪造安全检查证明。影响：工作流安全绕过。

涵盖八类共 25 种攻击变体的完整分类法（T-A 至 T-H）。第 8 节验证了其覆盖范围。